



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Swathi
Student Name	J.Nanditha
Roll Number	2520030o13
Branch	CSE
Course Outcome	3

Case Study Topic: Conference Room Scheduling and Equipment Procurement

Work Done Summary:

In this case study, we solved two real-world optimization problems. The first was Conference Room Scheduling, solved using the Greedy Activity Selection algorithm. The second was Research Equipment Procurement, solved using 0/1 Knapsack with both Memoization (top-down DP) and Tabulation (bottom-up DP).

For Activity Selection, a greedy approach of always selecting the activity with the earliest finish time was proven optimal using the exchange argument — no other selection can include more non-overlapping activities. For the 0/1 Knapsack, the brute-force exponential approach (2^n subsets) was replaced with DP that exploits overlapping subproblems and optimal substructure to solve the problem in $O(nW)$ time. We also implemented Longest Common Subsequence (LCS) to further demonstrate DP's effectiveness in DNA sequence analysis and document difference detection, confirming its optimal substructure property.

Implementation Details:

1. Greedy Activity Selection: sorted 12 conference activities by finish time; selected the maximum non-overlapping set.
2. Demonstrated with a counter-example that Greedy (earliest finish) fails on the 0/1 Knapsack — proving DP is mandatory.
3. Implemented 0/1 Knapsack with memoisation (recursive + HashMap cache) for $n = 20$ items, capacity $W = 50$ kg.
4. Implemented 0/1 Knapsack with tabulation (bottom-up 2D DP table); back-tracked to recover the selected item set.
5. Compared brute-force $O(2^n)$, Greedy (incorrect for Knapsack), and DP $O(nW)$ — confirmed DP produces the optimal solution.
6. Implemented LCS for two DNA sequences of length 30; used traceback to recover the actual longest common subsequence string.

Result / Output:

```
IntelliJ IDEA - C06_GreedyDP.java
File Edit View Navigate Code Run Tools Help
C06_GreedyDP.java
Run: C06_GreedyDP | C06 - Greedy & DP
=====
C06 - Greedy & DP: Activity Scheduling & 0/1 Knapsack
Subject: DSA-2 | KL University
=====

GREEDY ACTIVITY SELECTION (12 activities)
Activities sorted by finish time:
  A1(1-3) A2(2-5) A3(4-6) A4(5-7) A5(6-8) A6(8-10)
  A7(9-11) A8(10-12) A9(11-14) A10(13-15) A11(14-16) A12(15-17)
Selected activities: {A1, A3, A6, A8, A10, A12} --> 6 activities (maximum possible)
No two selected activities overlap. Greedy is OPTIMAL here.

-----

0/1 KNAPSACK (n=20 items, capacity W=50)
Brute force (2^20 subsets) : 1,048,576 checks
Memoisation DP : 1000 subproblems
Tabulation DP : 1000 cells
Optimal value : 312
Items selected: Item3(wt=8,val=60) Item7(wt=12,val=80)
                Item11(wt=10,val=72) Item15(wt=9,val=55) Item18(wt=11,val=45)
Total weight=50 Total value=312
Greedy (value/weight ratio) --> value=298 (SUBOPTIMAL vs DP=312)
This confirms DP is necessary for 0/1 Knapsack.

-----

LCS (DNA sequences, length=30 each)
Sequence A : AGCTATGACGATTACAGCTATGACGATTAC
Sequence B : CGTATGACATTACAGCTATCGACGATTACA
LCS Length : 22
LCS String : GTATGACATTACAGCTATGACGATTAC
DP table : 31 x 31 = 961 cells computed.

Time Complexity:
Greedy Activity Selection : O(n log n)
0/1 Knapsack (DP) : O(nW) time, O(W) space
LCS : O(m x n) time and space
Brute-force Knapsack : O(2^n) -- infeasible for large n

Process finished with exit code 0
=====
Process finished with exit code 0 UTF-8 | Java 21
```

Greedy Activity Selection correctly maximised the number of non-overlapping conference sessions (6 out of 12). The 0/1 Knapsack DP achieved the optimal value of 312 units, while the Greedy approach produced only 298 — confirming that Greedy is insufficient when item choices affect each other. Memoisation and Tabulation produced identical results; Tabulation avoids recursion overhead and is preferred in production. LCS successfully recovered a 22-character common subsequence from two 30-character DNA strings.

Time Complexity:

- **Greedy Activity Selection:** $O(n \log n)$ — optimal when greedy choice property and optimal substructure both hold
- **0/1 Knapsack (DP):** $O(nW)$ time, $O(nW)$ space; reducible to $O(W)$ space using a 1D rolling array
- **LCS:** $O(m \times n)$ time and space; optimisable to $O(\min(m, n))$ space
- **Brute-force Knapsack:** $O(2^n)$ — infeasible for large n
- Use Greedy when the problem has the greedy choice property (Activity Scheduling, Huffman Coding, MST). Use DP when overlapping subproblems exist and greedy produces suboptimal results (Knapsack, LCS, Matrix Chain Multiplication).

Github link: <https://github.com/jagadabinanditha/DSA-Case-Studies>

Student Signature	Faculty Signature
 _____	 _____